Ticket-Watcher

Thema: Geschäftsprozesse mit Camunda-Plattform

Dokumentinformationen

Dateiname: documentation.docx

Speicherdatum: 31.03.2026

Autoren: Luka Grujic, Elif Berra Canmaya

Inhaltsverzeichnis

Referenzdokumente 2
1 Einleitung 3
1.1 Auftragsbeschreibung 3
1.2 Setup 3
2 Vorgehensweise 5
3 Umsetzung 6
3.1 Architektur 6
3.2 Error-Handling 6
3.3 Tests 6
3.4 Technical Debt 6
4 Fazit 8
4.1 Schwierigkeiten 8
4.2 Reflektion 8

Referenzdokumente

[1] backend-structure.drawio

[2] todo-list_concert-ticket-watcher.txt

[3] StartNewArtistWatch.html

[4] FillInConcertSettingsForm.html

[5] RespondToInvitation.html

[6] ViewInvitationResults.html

1 Einleitung

1.1 Auftragsbeschreibung

Im Rahmen dieses betrieblichen Auftrags wurde eine automatisierte Engine entwickelt, die Nutzer benachrichtigt, sobald Konzerttickets für einen gewünschten Künstler in ihrer Nähe verfügbar sind. Die Idee entstand aus dem alltäglichen Problem, dass Konzerte oft ausverkauft sind, bevor man von deren Existenz erfährt. Die Engine übernimmt das kontinuierliche Überwachen und Prüfen vollständig automatisiert.

1.1.1 Zentral Anforderungen

Nr.	Anforderungen	Umsetzung durch
1	Standort des Nutzers automatisch ermitteln	GeolocationFetcher (ip-api.com)
2	Konzerte eines Künstlers in der Nähe suchen	TicketmasterEventFetcher
3	Eingeladene Nutzer validieren	InviteeTaskService
4	Ticket-Verfügbarkeit eines Events prüfen	TicketmasterTicketFetcher
5	Nutzerdaten persistent speichern	UserRepository (PostgreSQL)
6	Fehlgeschlagene Aufgaben automatisch wiederholen	Camunda handleFailure()

1.2 Setup

1

PostgreSQL Container starten

```
docker run --name concert-db -e POSTGRES_DB=concertdb -e
POSTGRES_USER=admin -e POSTGRES_PASSWORD=admin -p 5432:5432 -d
postgres
```

2

Camunda Container starten

```
docker run --name camunda -p 8080:8080 -d camunda/camunda-bpm
platform:7.20.0
```

3

Camunda öffnen und einloggen

http://localhost:8080/camunda (Username: demo | Password: demo)

4

BPMN deployen

Camunda Modeler öffnen, ConcertTicketWatcher.bpmn laden und zusammen mit allen. form-Dateien auf *http://localhost:8080/engine-rest* deployen

5

Benutzer anlegen

Gehe zu *http://localhost:8080/camunda/app/admin*. Mindestens zwei Benutzer mit E-Mail-Adresse erstellen. (Das wird später wichtig, um das Einladungssystem zu benutzen)

6

Java-Projekt Vorbereiten

Öffne das projekt mit IntelliJ. Im Projekt hat es eine Datei config.example.properties in *src/main/resources/*. Kopiere diese und in bennene sie in config.properties um. Anschliessend trage dein Ticketmaster API-Key sowie die Gmail-Adresse und das Gmail App-Passwort ein.

API Key: *https://developer.ticketmaster.com/products-and-docs/apis/getting-started/*

Gmail Konto: Dafür kannst du dein eigens verwenden. Jedoch beim Passwort wird der App-Passwort verlangt, den du zuerst in den Einstellungen erstellen must. Ebenfalls musst du 2-Faktor-Autetifizierung aktiviert haben.

7

Java-Projekt starten

IntelliJ öffnen -> Gradle-Dependencies neu laden -> X ->Main.java ausführen

8

Prozess starten

http://localhost:8080/camunda/app/tasklist öffnen und den Prozess starten.

9

Fertig

Von hier an sollte der Concert Ticket Watcher laufen und selbsterklärend sein. Falls nicht, sind noch schönere Forum-Designs in den Anhängend, die im Projekt leider nicht implementiert werden konnten, aufgrund des Gebrauchs einer älteren Camunda Version.

2 Vorgehensweise

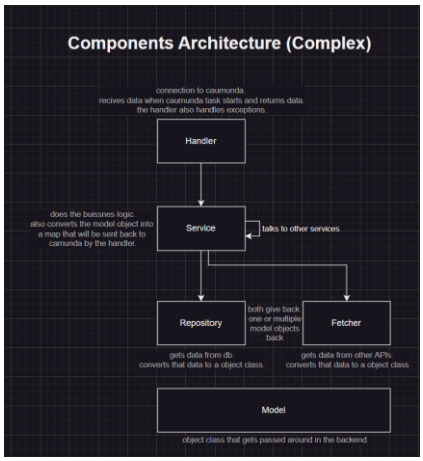
Das Projekt wurde in mehreren Phasen durchgefuehrt. In der ersten Phase wurde die Architektur des Prozesses mit dem Camunda Modeler definiert und iterativ verfeinert. Parallel dazu wurden die Anforderungen an das Java-Backend festgelegt. In der zweiten Phase erfolgte die schrittweise Implementierung, beginnend mit den abstrakten Basisklassen der generischen Schicht, gefolgt von den konkreten Fetchers, Repositories, Services und Handlers. In der dritten Phase wurde das Backend mit dem Camunda-Prozess verbunden und end-to-end getestet. Als gemeinsames Planungswerkzeug haben wir eine txt Todo-Liste erstellt (im Anhang) und auf gute Kommunikation vertraut.

3 Umsetzung

3.1 Architektur

Das Backend ist nach einem Spring-Boot-inspirierten Schichtmodell aufgebaut, ohne Spring Boot selbst zu verwenden. Es besteht aus vier Schichten: Handlers empfangen Aufgaben von Camunda und delegieren sie an Services. Services enthalten die eigentliche Geschäftslogik und rufen Fetchers auf. Fetchers sind für externe API-Aufrufe zuständig und geben typisierte Modelle zurück. Repositories verwalten den Datenbankzugriff. Die gesamte Verdrahtung der Abhängigkeiten erfolgt manuell in der Setup-Klasse. Eine Übersicht der Architektur ist im Referenzdokument backend-architektur.drawio zu finden.

Zudem sollten sowohl das BPMN-Diagramm als auch das Backend ausreichend mit kommentaren verseht sein, die weit ins Detail spezifische Elemente erklären.



3.2 Error-Handling

Das System unterscheidet zwischen fatalen Fehlern und wiederkehrenden Fehlern. Fatale Fehler, wie ein nicht erreichbarer Geolocation-Dienst oder ein Datenbankfehler, führen zu einem sofortigen Abbruch des Prozesses. Wiederkehrende Fehler, wie ein nicht gefundenes Konzert oder noch nicht verfügbare Tickets, werden stillschweigend protokolliert und beim nächsten Zyklus erneut versucht. Der Nutzer wird bei einem Absturz des Prozesses nicht direkt informiert, da kein eigenes Benachrichtigungssystem für Fehler implementiert wurde in Camunda. Jedoch das Backend informiert über jeden Error.

3.3 Tests

Auf formelle Unit-Tests wurde verzichtet. Das System wurde stattdessen durch wiederholtes manuelles Durchspielen des gesamten Prozesses getestet, einschliesslich gezielter Edge Cases wie ungültige Benutzernamen oder nichtexistierende Künstler. Da die internen Polling-Zyklen des Systems 6 bis 12 Stunden betragen, wurde eine separate Testversion des Systems erstellt, die Konzert- und Ticketdaten sofort zurückgibt, ohne auf echte Ticketmaster-API-Daten zu warten.

3.4 Technical Debt

Da es sich um ein Schulprojekt mit begrenztem Zeitrahmen handelt, wurden Technical Debts bewusst in Kauf genommen.

3.4.1 Silent Failure

Ungültige oder nicht registrierte Einladungs-Benutzernamen werden stillschweigend ignoriert, ohne den Benutzer zu informieren. Der Concert Ticket Watcher wird nicht crashen, dies ist weiterhing robust.

3.4.2 Alles nur auf einem Computer

Das gesamte System läuft auf einer einzigen lokalen Maschine. Kommunikation zwischen mehreren Computern wird nicht unterstützt.

3.4.3 Nicht das effizienteste Backend

Kein Global-Class-Handler: Alle Klassen müssen in Setup.java manuell instanziiert und injiziert werden. Es gibt kein automatisches Dependency-Injection-System wie in Spring Boot, das Klassen selbstständig erkennt und verdrahtet.

Kein DTO-System: Input und Output werden als einfache String-Schlüssellisten definiert. Es gibt keine Garantie zur Kompilierzeit, dass die Schlüssel mit den BPMN-Prozessvariablen übereinstimmen.

Schlechtes Error-Handling: Fast alle Fehler werden als generische RuntimeException weitergegeben. Es wird ebenfalls nicht zwischen behebbaren und nicht behebbaren Fehlern unterschieden.

Keine Unit-Tests wurden geschrieben.

4 Fazit

4.1 Schwierigkeiten

Eine der groessten Hürden war das Deployment des BPMN-Prozesses in Camunda 7.20. Verschiedene Formular-Komponenten wie HTML-Felder und Bild-Elemente, die im Camunda Modeler ohne Fehlermeldung erstellt werden können, werden von der Camunda-7.20-Engine zur Laufzeit nicht unterstützt. Dies führte zu wiederholten Deployment-Fehlern, die nur durch manuelles Bereinigen der Formulardateien behoben werden konnten. Zudem war das Debugging des Zusammenspiels zwischen dem Java-Backend und der Camunda-Engine aufwendig, da Fehler in Service Tasks den Prozess still stoppen können, ohne eine klar sichtbare Fehlermeldung zu erzeugen.

4.2 Reflektion

Das Projekt hat gezeigt, dass Camunda eine leistungsfähige Plattform für die Automatisierung von Geschäftsprozessen ist, die jedoch eine steile Lernkurve mit sich bringt. Die Kombination aus BPMN-Prozessmodellierung und externem Java-Backend hat sich als sinnvoll erwiesen, da beide Teile unabhängig voneinander entwickelt und getestet werden konnten. Besonders wertvoll war die Erkenntnis, dass ein klar strukturiertes Backend mit definierten Schichten die Entwicklung erheblich erleichtert und spätere Erweiterungen vereinfacht. Insgesamt war das Projekt trotz der technischen Schwierigkeiten eine lehrreiche Erfahrung im Umgang mit prozessgetriebenen Systemen.